CityPulse: NYC Taxi Mobility Explorer

Technical Report: UrbanMobilityDataExplorer
NYC TLC Yellow Taxi Trip Records, January 2019

1. Problem Framing and Dataset Analysis

The New York City Taxi & Limousine Commission (TLC) publishes a record of every yellow taxi trip. We worked with the January 2019 file, which contains 7,696,617 trips. Each trip has pickup and dropoff timestamps, trip distance, passenger count, payment type and an itemised fare. Two supporting files complete the data. The taxi zone lookup table maps each of the 265 location IDs to a borough and zone name. The taxi zones file holds the boundary of each zone. Together they form a small star schema, with the trips as the fact table and the zones as the dimension.

The raw data has several quality problems. We cleaned it with ten rules applied in a fixed order. Each removed row was logged under the first rule it broke, so every row is counted exactly once. In total 394,574 rows (5.1%) were removed. Table 1 lists the rules and the assumption behind each one.

Rule	Rows removed	Share	Assumption
Unknown zone	187,399	2.43%	Zones 264 and 265 are "Unknown" and cannot be mapped
Passenger count	107,166	1.39%	0 or more than 6 passengers is a data-entry error
Distance	37,578	0.49%	0 miles, or over 100 miles, is not a city taxi trip
Missing values	28,672	0.37%	All were a missing passenger count
Duration	20,033	0.26%	Trips over 6 hours are meters left running
Fare	8,299	0.11%	Fares of $0 or less (refunds) or above $500
Speed	4,599	0.06%	An average above 80 mph is impossible in the city
Date out of range	524	0.01%	The January file contained trips from Nov/Dec 2018
Negative duration	297	<0.01%	Dropoff recorded at or before pickup
Duplicate	7	<0.01%	Identical timestamps, zones, fare and distance

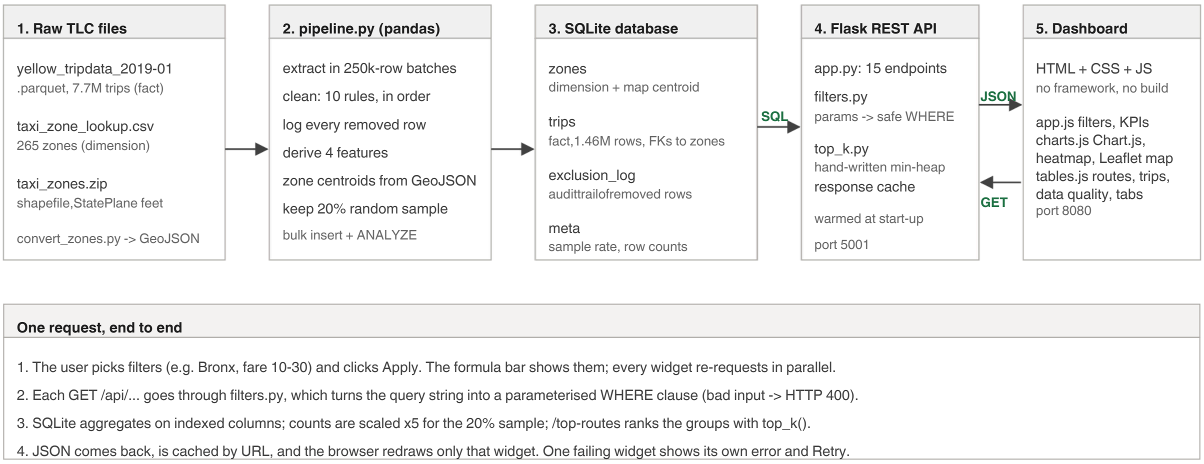
Table 1: Cleaning rules, in the order they are applied

After cleaning, we standardised the remaining data. Timestamps are stored in ISO format, and zone IDs and passenger counts as integers. We also stored the pickup hour, weekday and date as separate columns so they never have to be parsed again. We derived four features from the raw columns:

- **Average speed (mph):** distance divided by duration. We use it as a measure of congestion.
- **Fare per mile:** shows how the minimum fare makes short trips expensive.
- **Tip percentage:** tip as a share of the fare, for card payments only.
- **Duration in minutes:** needed for the other two.

Unexpected observation. While exploring the data we found that cash trips record an average tip of exactly $0.00, against $2.52 for card trips. This affects about 2 million trips. The meter cannot see cash tips, so the zero means "not recorded", not "no tip". An average tip that included cash trips would be badly understated. We therefore calculate the tip percentage only for card trips, and the dashboard labels it as such. We also kept some records that pass every rule but still look wrong, such as a $100 fare for 0.01 miles. A fare-per-mile rule is left as future work.

2. System Architecture and Design Decisions

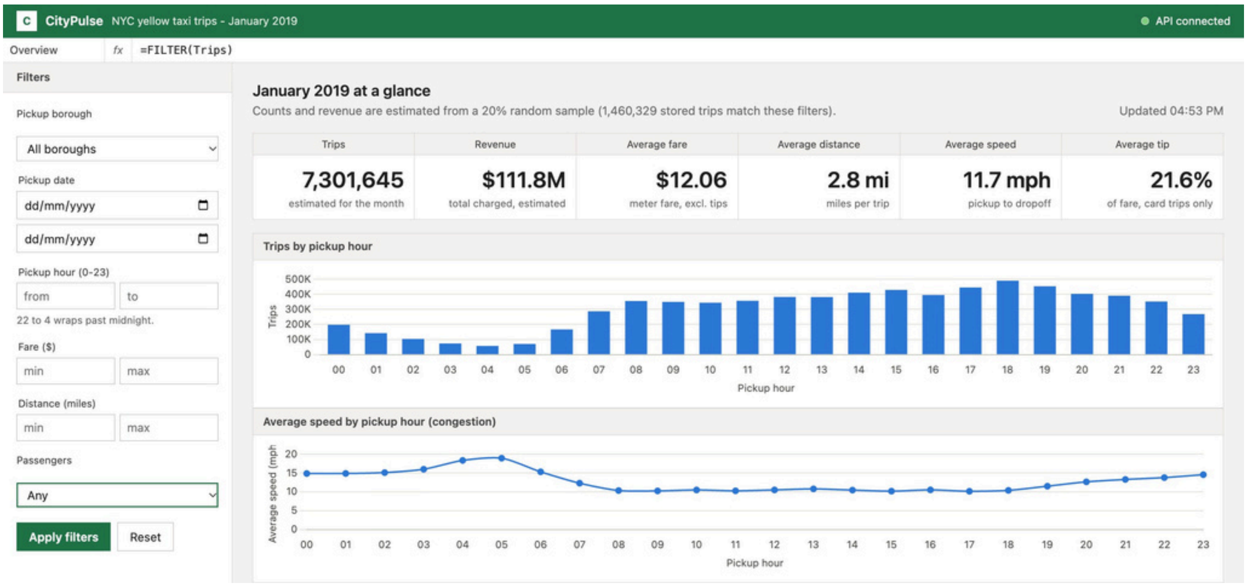


1. The user picks filters (e.g. Bronx, fare 10-30) and clicks Apply. The formula bar shows them; every widget re-requests in parallel.
2. Each GET /api/... goes through filters.py, which turns the query string into a parameterised WHERE clause (bad input -> HTTP 400).
3. SQLite aggregates on indexed columns; counts are scaled x5 for the 20% sample; /top-routes ranks the groups with top_k().
4. JSON comes back, is cached by URL, and the browser redraws only that widget. One failing widget shows its own error and Retry.

Figure 1: System architecture, from the raw files to the dashboard

The system has five parts (Figure 1). A Python pipeline using pandas reads the Parquet file in batches of 250,000 rows, so memory use stays bounded. It cleans the rows, derives the features and loads them into SQLite. We chose SQLite because it is a single file with no setup, which suits a read-only analytical workload. A Flask REST API with 15 endpoints serves the data. The front end is plain HTML, CSS and JavaScript with Chart.js and Leaflet, so there is no build step. The zone shapefile uses New York State Plane coordinates. A small script reprojects it to latitude and longitude, and the pipeline averages each zone's boundary points to place it on the map. This is how the trips are linked to the spatial data.

The database schema is normalised. The *zones* table holds each zone and its map position. The *trips* table references zones through two foreign keys, one for pickup and one for dropoff, and is indexed on the columns we filter by most: pickup and dropoff zone, date, hour, timestamp and fare. An *exclusion_log* table keeps the record of removed rows, and a small *meta* table stores facts about the pipeline run. All user input is passed to SQL as bound parameters, which prevents SQL injection.



NYC yellow taxi trips - January 2019

=FILTER(Trips)

Pickup borough: All boroughs
Pickup date: dd/mm/yyyy
dd/mm/yyyy
Pickup hour (0-23): from to
22 to 4 wraps past midnight.
Fare ($): min max
Distance (miles): min max
Passengers: Any

January 2019 at a glance

Counts and revenue are estimated from a 20% random sample (1,460,329 stored trips match these filters).

Updated 04:53 PM

Trips	Revenue	Average fare	Average distance	Average speed	Average tip
7,301,645 estimated for the month	$111.8M total charged, estimated	$12.06 meter fare, excl. tips	2.8 mi miles per trip	11.7 mph pickup to dropoff	21.6% of fare, card trips only

Trips by pickup hour

Average speed by pickup hour (congestion)

Figure 2: The dashboard with filters (left), summary figures, trips by hour and speed by hour

We made three main trade-offs:

1. **We sampled the data.** With all 7.3 million clean trips (1.5 GB), each chart took 20 to 120 seconds on our 8 GB laptops. The pipeline still cleans every row, so the cleaning statistics are exact. It then stores a random 20% sample (1.46 million trips) using a fixed seed, and the API multiplies counts and totals by five. The estimated total of 7,301,645 trips is within 0.005% of the true 7,302,043.
2. **We tuned queries instead of adding hardware.** Running SQLite's ANALYZE command reduced one filtered query from 79 seconds to 0.03 seconds, and changing a join order reduced the sorted trip list from 45 seconds to 1 millisecond. The API also caches each response, which is safe because the data only changes when the pipeline is run again.
3. **We aggregate on the server.** Every number on the dashboard is calculated in SQL with the user's filters applied, rather than in the browser from a sample. This needs more queries, but the numbers always agree with each other.

3. Algorithmic Logic and Data Structures

The dashboard shows the ten busiest routes, where a route is a pickup zone and a dropoff zone. SQL groups the trips into 19,769 routes, and we need only the top ten. Instead of sorting all of them, we wrote our own binary min-heap (file top_k.py), without heapq, sorted() or any other built-in sorting. The heap is stored in an array: the parent of position i is at (i - 1) / 2, and its children are at 2i + 1 and 2i + 2. The heap keeps the k best routes found so far, with the weakest of them at the root, so each new route only needs to be compared with one value. The routes are read one at a time straight from the database.

```
top_k(items, k):
    if k <= 0: return empty list
    heap = empty min-heap ordered by trip count
    for each item in items:
        if size(heap) < k:
            push(item)
        else if item.count > peek().count:
            pop()
            push(item)
    result = list of size(heap)
    for slot from last to first:
        result[slot] = pop()          # the heap gives the smallest first
    return result                     # so the result is largest first
```

Complexity. Each route costs at most one pop and one push on a heap of size k, so the running time is O(n log k) and the extra memory is O(k). A full sort would be O(n log n) time and O(n) memory. With k = 10 and n = 19,769, log k is about 3.3 while log n is about 14.3.

In practice, our heap took 4.0 ms on the real routes and Python's built-in sort took 2.1 ms, with identical results. The built-in sort is written in C, so at this size it wins on speed despite the worse complexity. The advantages of the heap are that it never holds all the routes in memory at once, and that its cost grows with k rather than with the total number of routes. We tested it with 12 unit tests: ordering, empty input, k of zero or less, k larger than the input, ties, and 200 random lists checked against Python's sort.

4. Insights and Interpretation

Insight 1: Rush hour halves taxi speed on weekdays

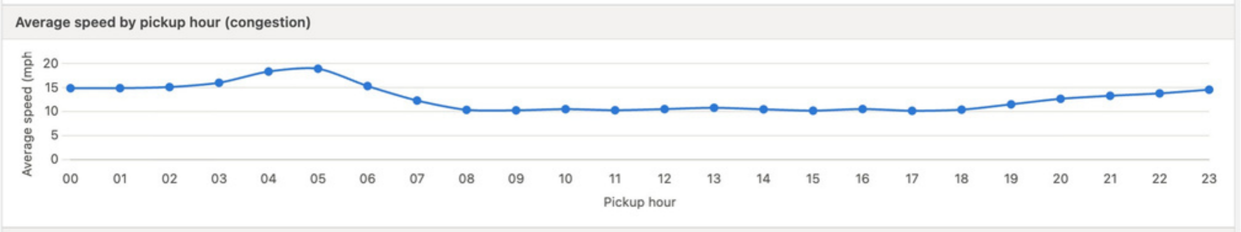
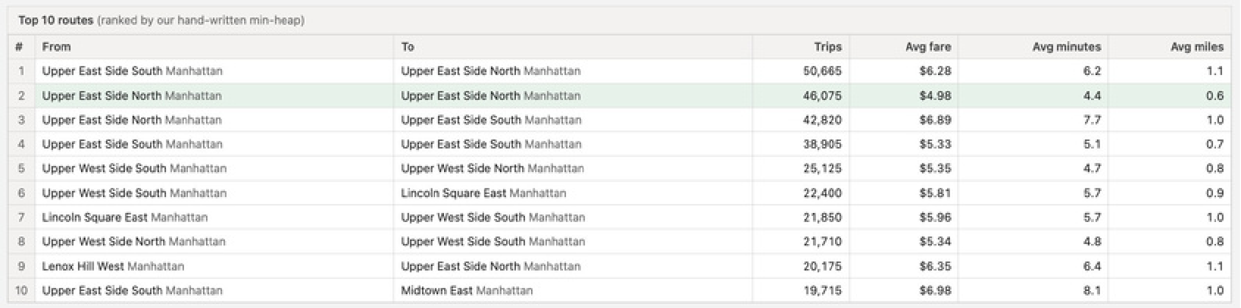


Figure 3: Average trip speed by pickup hour

We averaged the derived speed feature for each pickup hour and compared weekdays with weekends. On weekdays, taxis average 18.7 mph at 5 am but only 9.5 mph at 8 am and 9.9 mph at 5 pm. On weekends at 8 am they still average 15.9 mph. The slowdown follows commuter demand, not the time of day itself. The same trip takes about twice as long in the weekday rush, which raises time-based fares and matters for congestion pricing.

Insight 2: Taxi demand is short trips within Manhattan



Top 10 routes (ranked by our hand-written min-heap)

#	From	To	Trips	Avg fare	Avg minutes	Avg miles
1	Upper East Side South Manhattan	Upper East Side North Manhattan	50,665	$6.28	6.2	1.1
2	Upper East Side North Manhattan	Upper East Side North Manhattan	46,075	$4.98	4.4	0.6
3	Upper East Side North Manhattan	Upper East Side South Manhattan	42,820	$6.89	7.7	1.0
4	Upper East Side South Manhattan	Upper East Side South Manhattan	38,905	$5.33	5.1	0.7
5	Upper West Side South Manhattan	Upper West Side North Manhattan	25,125	$5.35	4.7	0.8
6	Upper West Side South Manhattan	Lincoln Square East Manhattan	22,400	$5.81	5.7	0.9
7	Lincoln Square East Manhattan	Upper West Side South Manhattan	21,850	$5.96	5.7	1.0
8	Upper West Side North Manhattan	Upper West Side South Manhattan	21,710	$5.34	4.8	0.8
9	Lenox Hill West Manhattan	Upper East Side North Manhattan	20,175	$6.35	6.4	1.1
10	Upper East Side South Manhattan	Midtown East Manhattan	19,715	$6.98	8.1	1.0

Figure 4: The ten busiest routes, ranked by our min-heap

The top routes come from our min-heap (Section 3). 92.7% of all pickups happen in Manhattan. The ten busiest routes are all inside Manhattan, mostly within the Upper East and Upper West Sides, and each averages at most 1.1 miles, 4 to 8 minutes and $5 to $7. Across all trips, journeys under one mile make up 27.3% of rides and cost $12.93 per mile, compared with $3.00 per mile for trips over ten miles. Yellow taxis mostly serve very short trips in dense residential areas, distances that could often be walked, cycled or covered by bus. These corridors are where bike share and bus improvements would have the most effect.

Insight 3: Weekdays and weekends have different demand patterns

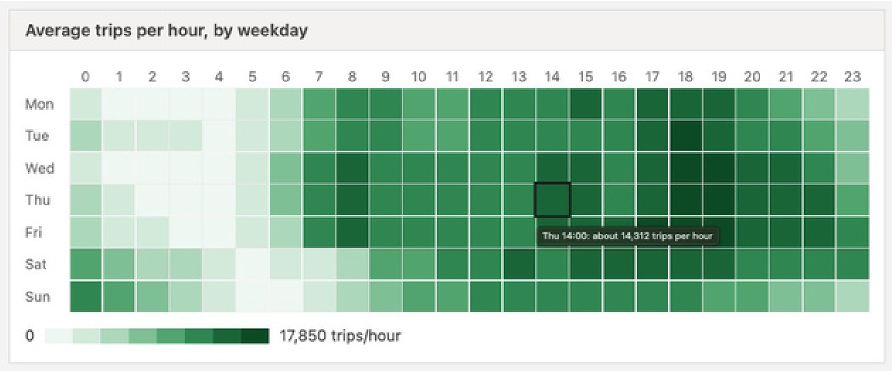


Figure 5: Average trips per hour for each weekday and hour

The heatmap shows the average number of trips in each hour of each weekday. At 8 am, weekdays average 13,495 trips per hour against 5,497 at weekends, about 2.5 times as many. At 1 am the pattern reverses: weekends average 9,072 trips per hour against 3,024 on weekdays. The hours from midnight to 4 am account for 14.5% of weekend trips but only 4.7% of weekday trips. The busiest hour of the month is Friday at 6 pm, with 17,850 trips per hour. On weekdays, taxis serve commuters. At weekends, they replace public transport late at night, when subway service is less frequent. Fleet planning and driver shifts should follow these two patterns rather than a single daily average. (January 1st fell on a Tuesday, which makes the early hours of Tuesday look busier than usual.)

5. Reflection and Future Work

Technical challenges. The biggest problem was performance. The full month was too large to query interactively on our laptops, which led to the sampling and query tuning described in Section 2. The zone shapefile used State Plane coordinates rather than latitude and longitude, so it had to be reprojected before it could be mapped. The order of the cleaning rules also mattered, because a single row can break several rules. For example, a row with 0 passengers and 0 miles is counted once, under the first rule it breaks.

Team and process challenges. Our first plan had eight stages and would have taken weeks, so we cut it to what the assignment requires and built it in three days. Most debugging time went into small typing mistakes. Errors that crashed the program were easy to find. The harder ones failed silently: swapped latitude and longitude, a wrong index in the heap, AND written instead of OR in a filter, and styling that did not apply. We learned to use a code checker (pyflakes), to write unit tests (they caught the heap bug), and to check each layer against known numbers. We also nearly uploaded a 372 MB database to GitHub before adding a .gitignore file, which taught us to check the repository contents before pushing.

Future work. If this were a real product, we would:

- load all months, and move to PostgreSQL or DuckDB with pre-aggregated tables so that no sampling is needed;
- add a fare-per-mile rule for implausible fares, and detect duplicates across the whole file rather than within each batch;
- add user accounts, restrict which websites can call the API, and run it on a production server; compare
- months and weather conditions, and alert city planners when travel patterns change.